Phase 2: Simulator and verification

Fault tolerant control and post fault trajectory planning of a redundant cable driven parallel robot

M. J. Tachia, A. V. Maloletov

Technical report, 26 October 2026

About this document. This document describes the simulator on which every result of the study will be produced and reports the verification gates of the specification. It builds on the Phase 1 methods (`docs/phase1/methods.pdf`), whose equation numbers are referred to as (M n). Two changes to the Phase 1 model came out of this phase and are already carried into the methods: the orientation is integrated as a unit quaternion, and the damping of a cable fades to zero at zero stretch. One departure from the specification is made on evidence: the plant step is 62.5 μs, below the range of 0.1 to 0.5 ms written in the specification, because the step convergence test fails above it once a cable breaks. The closed loop runs here use a verification controller, a motor angle PD with tension feedforward, which is not the method of the study; its only purpose is to exercise the simulator before Phase 3. All results carry the code digest `c77663b917040fde`.

1 Simulation model

The simulator integrates the full model of Phase 1, the cable law (M 13) and the winch equation (M 17): a rigid platform with six degrees of freedom, eight elastic cables that can only pull, and eight geared winches whose motors are driven by current controlled drives. Three additions turn that model into a simulator that can be trusted through a fault, and a fourth defines what the controller is allowed to see.

Orientation. The plant state is

$$\boldsymbol{x} = \left[\boldsymbol{p}^\top,\ \boldsymbol{\eta}^\top,\ \boldsymbol{v}^\top,\ \boldsymbol{\omega}^\top,\ \boldsymbol{\theta}^\top,\ \dot{\boldsymbol{\theta}}^\top\right]^\top \in \mathbb{R}^{13+2n}, \qquad \dot{\boldsymbol{\eta}} = \tfrac{1}{2}\begin{bmatrix} -\boldsymbol{\omega}^\top \boldsymbol{\epsilon} \\ \eta_0 \boldsymbol{\omega} + \boldsymbol{\omega} \times \boldsymbol{\epsilon} \end{bmatrix}, \tag{1}$$

where $\boldsymbol{\eta} = [\eta_0, \boldsymbol{\epsilon}^\top]^\top$ is the unit quaternion of the platform and $\boldsymbol{\omega}$ its angular velocity in the world frame. The quaternion is renormalised after every integration step. The Euler angles of Phase 1 have a singularity at a pitch of $\pm 90°$, which a controlled platform never approaches but an uncontrolled one can reach after an upper cable breaks, as the runs of Section 3.1 show; with the quaternion the plant stays regular through any rotation, and the ZYX angles are recovered from $\boldsymbol{\eta}$ for the estimators, the controller and the logs. Everything else in the platform dynamics is (M 12) unchanged.

Cables near slack. A cable becomes slack whenever its stretch reaches zero, and after a fault this happens many times, as cables lose their load and catch it again. In the plain Kelvin–Voigt law a cable that tightens at the stretch rate $\dot{\Delta}$ jumps at once to the force $\beta k \dot{\Delta}$, which is neither physical nor harmless for a fixed step integrator, since a jump inside a step costs the integrator its order. The tension law (M 13) therefore carries the factor $s(\Delta) = 3r^2 - 2r^3$, $r = \min(\Delta/\Delta_s, 1)$, which lets the damping grow from zero with the stretch, as Hunt and Crossley proposed for impacts [Hunt and Crossley, 1975]. With $\Delta_s = 0.1$ mm the factor equals one above a tension of 0.4 to 2.2 N, depending on the cable length, so every taut state of the robot follows the identified law. The effect on the solution is not small: in the stiffest scenario of Section 3.1, the peak tension after a lower cable breaks is 244 N with the factor and 360 N without it, everything else being equal, so the extra 116 N comes from the jump alone.

Drive limits. The drive applies the commanded torque τ_i only up to the peak torque of the motor, and while the motor is driving ($\tau_i \dot{\theta}_i > 0$) it lets the torque fall linearly to zero over the last 10% below the maximum speed,

$$\tau_i^{\mathrm{drv}} = \begin{cases} \bar{\tau}_i \, \min\Big(1, \dfrac{\dot{\theta}_{\max} - |\dot{\theta}_i|}{0.1\,\dot{\theta}_{\max}}\Big)^+, & \bar{\tau}_i \dot{\theta}_i > 0, \\ \bar{\tau}_i, & \text{otherwise}, \end{cases} \qquad \bar{\tau}_i = \mathrm{sat}_{\tau_{\mathrm{pk}}}(\tau_i), \tag{2}$$

Table 1: Sensor model and loop timing. The resolution is the manufacturer's example for the M1 option; the current noise is a design choice evaluated at one and two times its value.

Quantity	Value	Source or status
Encoder	M1: Hiperface, multi turn (4096 rev.), 128 signal periods	Bosch Rexroth AG [2018], Sec. 4.3
Resolution	2^{20} counts/rev (25 nm of cable)	manufacturer's example; depends on the drive
Accuracy	$\pm 120''$, once per revolution, random phase	Bosch Rexroth AG [2018]; error shape our choice
Torque measurement	τ^{drv} + noise, $\sigma = 0.004$ N m (0.96 N)	design choice; evaluated at 1× and 2×
Delay	one inner sample (1 ms)	specification
Inner rate	1 kHz, zero order hold	2 ms set points on IPAnema [Pott et al., 2013], 1 ms on IPAnema 3 [Kraus et al., 2015]
Plant step	62.5 μs, RK4, 16 steps per sample	step convergence test (Section 3.1)

with $\tau_{\mathrm{pk}} = 4.0$ N m and $\dot{\theta}_{\max} = 9000$ rpm for the MSK030C [Bosch Rexroth AG, 2018]. The faults (M 18) act after the drive, $\tau_i^{\mathrm{act}} = (1-\rho_i)\tau_i^{\mathrm{drv}} + f_i$, so the drive's own current, and with it the measured torque, stays τ_i^{drv} whatever happens downstream. The speed limit matters after a break: the winch of the broken cable loses its load, and a tension loop that keeps asking for its reference would otherwise spin the motor without bound. The shape of the limiter is our choice, since the limiter of the drive is not documented.

Drum windows. The drum holds 6 m of cable, which the methods turned into a length window $l_i^- \le l_{u,i} \le l_i^- + \Delta l_{\max}$ for each cable (M 15). The plant does not stop at the window but flags every sample at which a drum would be overfull or empty, so that a run that leaves the windows is recorded as such.

Sensors and timing. Each winch reports two signals, the motor angle from its multi turn absolute encoder and the motor current, and no cable carries a load cell. Table 1 lists the sensor model. The encoder option fitted to IPAnema 2 is not published; we take the M1 option of the MSK motors, which is multi turn and absolute as Pott et al. [2013] state, and its resolution of $128 \times 2^{13} = 2^{20}$ counts per revolution, the figure that the manufacturer's own example gives for this option [Bosch Rexroth AG, 2018]. Through the gearbox one count is 25 nm of cable, so the quantisation is far below anything the platform could resolve, but it is not irrelevant to the observer of each winch, whose third channel differentiates the angle three times. The system accuracy of $\pm 120''$ enters as an error that repeats once per motor revolution with a random phase per winch,

$$\tilde{\theta}_i = q_e \Big\lfloor \frac{\theta_i + a_e \sin(\theta_i + \varphi_i)}{q_e} \Big\rceil, \qquad q_e = \frac{2\pi}{2^{20}}, \quad a_e = 120'', \tag{3}$$

and the measured torque is $\tilde{\tau}_i = \tau_i^{\mathrm{drv}} + \nu_i$ with white Gaussian noise ν_i. No noise figure has been published for the current measurement of the drive, so its standard deviation, 0.004 N m or 0.96 N of cable tension, is a design choice that the evaluation doubles, as the specification asks.

At every inner sample $t_k = k \cdot 1$ ms the simulator first switches on any fault whose onset has been reached, then reads the encoders and the drive torque from the true state, hands the controller the reading taken one sample earlier, receives the torque command, and integrates the plant to t_{k+1} with the command held, in 16 RK4 steps. Faults therefore start on the millisecond grid, which is the resolution at which the controller could notice them in any case.

2 Implementations

The plant exists twice. The NumPy implementation follows the equations line by line and is the reference that the tests check. The compiled implementation, written with CasADi [Andersson et al., 2019], computes the same quantities and is the one that runs the campaign. It solves the twist equations in closed form instead of factorising $\mathbf{M}_\nu$: subtracting $[\boldsymbol{c}_w]_\times$ times the first block row of (M 12) from the second gives

$$\mathbf{I}_c\,\dot{\boldsymbol{\omega}} = \boldsymbol{r}_\omega - \boldsymbol{c}_w \times \boldsymbol{r}_v, \qquad \dot{\boldsymbol{v}} = \boldsymbol{r}_v/m + \boldsymbol{c}_w \times \dot{\boldsymbol{\omega}}, \tag{4}$$

where $\boldsymbol{r}_v$ and $\boldsymbol{r}_\omega$ are the force and moment on the right of (M 12) minus the bias terms, and $\mathbf{I}_c^{-1} = \mathbf{R}\mathbf{I}_G^{-1}\mathbf{R}^\top$ is available without inversion at run time. One call advances the state over a whole millisecond, so Python

Table 2: Verification gates on the study parameters. Gates marked S are those of the specification; the others were added in this phase.

Check	Criterion	Result
S Length Jacobian against central differences	rel. error $< 10^{-7}$	9.0×10^{-10} (Phase 1)
S Energy, locked winches, undamped cables, no input, 10 s at 62.5 μs	drift $< 10^{-6}$ of initial kinetic energy	3.5×10^{-11}
S $\dot{\mathbf{M}} - 2\mathbf{C}$ skew symmetric, $\dot{\mathbf{M}}$ exact	$< 10^{-10}$	7.1×10^{-15}
S Static equilibrium at home, study parameters	matches the LP	tensions 151.4 / 58.6 N, $\max \lvert\dot{\boldsymbol{x}}\rvert = 4.8 \times 10^{-10}$
S Step halving	metrics change $< 0.1\%$	passes at 62.5 μs; one exception (Section 3.1)
S Reference solution (Radau IIA, rtol 10^{-9})		5.8×10^{-8} m, 0.010 N after a break
S MuJoCo, same inputs, 5 s	difference $< 1\%$ of the motion	1.4×10^{-10} of the motion, with slack cables
Compiled plant against NumPy plant, 200 states		5.0×10^{-13} relative
Prediction model Jacobians against central differences	abs. error $< 10^{-7}$	pass
Sensor model, loop delay, fault onset, drive limits, drum flags	exact values	pass

is entered once per inner sample. Over 200 random states with large rotations, slack and broken cables, saturated drives and winches near their speed limit, the two implementations agree to a relative difference of 5×10^{-13} in $\dot{\boldsymbol{x}}$, and over a closed loop run without sensor noise their trajectories agree to round off. The compiled plant runs a 16 s closed loop run on the reference T1 in 11.7 s; the NumPy plant, at the 1.7 ms it needs per RK4 step, would take about seven minutes.

The estimators and the MPC need a third model, the rigid cable platform of (M 12) with the tensions as inputs. It is built in CasADi as well, discretised by one RK4 step, and returns its Jacobians with respect to the state and the tensions by algorithmic differentiation; the tests compare them with central differences and the dynamics with the NumPy rigid body model. The same module provides the unstretched lengths $EAl/(EA + t)$ that the encoders measure under a given tension, which is the measurement function of the IMM (M 29).

3 Verification

Table 2 collects the gates of the specification and adds the ones this phase introduced. All of them pass on the study parameters, with one qualification in the step convergence test that the following subsection explains.

3.1 Step convergence

The convergence test runs the closed loop on T1 for 8 s with the sensors on and repeats it with the plant step halved from 1 ms down to 31.25 μs. Five scenarios cover the range of what the campaign will ask of the integrator: no fault; a loss of effectiveness of 60% on winch 3 at 4 s; a break of the lower cable 5 at 4 s; a break of the upper cable 1 at 4 s; and a break of cable 5 with twice the nominal stiffness and the larger retardation time of 0.03 s, which are the fastest cable dynamics in the robustness ranges. The two most violent cases are also run without sensor noise and quantisation, so that the integrator can be judged apart from the sensitivity of the loop to single encoder counts. The metrics compared on halving are those the evaluation will report: the RMS and largest position error, the largest tilt, the largest and the mean tension, the share of time with a tension outside $[t_{\min}, t_{\max}]$, and the final position.

Without a fault the test is uneventful. Every metric changes by less than 0.03% at every step, and so does every metric of the actuator fault and of the lower cable break, although in the latter the platform tilts by 15° and at least one cable is outside its bounds for 35% of the run. The difficulty sits entirely in the peak tension after a break, because a cable that catches its load again produces a short tension peak whose height depends on where the integration step falls relative to the instant of tightening. Halving the step from 125 to 62.5 μs still changes that peak by 0.27% after the upper cable breaks (0.35% without noise) and by 0.15% in

the stiff case without noise; halving it from 62.5 to 31.25 μs changes it by less than 0.03% in every scenario except one. That exception is the upper cable break without noise, at 0.12%. In that run the verification controller, which does not know about the fault, lets the platform turn over, at least one cable is outside its bounds for half of the run, and the peak tension reaches 537 N, more than two and a half times $t_{\max}$. A run like that is a lost platform whatever the third digit of its peak tension, and we accept the 0.12% rather than halve the step once more, which would double the cost of every run in the campaign. The plant step is therefore 62.5 μs.

The pointwise histories converge more slowly than the metrics, and we report them for that reason. Between 62.5 and 31.25 μs the position histories agree within 37 μm in the worst case, the upper cable break without noise, while the tension histories differ by up to 2.4% of $t_{\max}$ at single samples during the snap sequences after the upper cable breaks. Pointwise tension histories during such sequences are therefore not used as evidence; the metrics above are.

3.2 Reference solution

Without sensor noise, the closed loop of the convergence test was solved a second time with the implicit Radau IIA method [Hairer and Wanner, 1996] of SciPy [Virtanen et al., 2020], with relative tolerance 10^{-9}, absolute tolerance 10^{-12}, the exact Jacobian of the compiled plant, and a restart at every inner sample. Over 6 s without a fault, RK4 at any of the steps tested agrees with it to 10^{-13} m. With the lower cable breaking at 4 s, RK4 at 62.5 μs stays within 5.8×10^{-8} m and 2.3×10^{-7} rad of the reference, and its tensions within 0.010 N, which is three orders of magnitude finer than anything the evaluation reports.

3.3 Independent cross-check in MuJoCo

The plant and MuJoCo [Todorov et al., 2012] share no code, so their agreement checks the rigid body dynamics, the cable kinematics and the unilateral cable law together. With the winches locked and the cables undamped, each cable is a spring of stiffness $EA/l_{u,i}$ that pulls only when stretched, and a spatial tendon in MuJoCo with the dead band $[0, l_{u,i}]$ on its spring length behaves in exactly that way. Both codes were started from the equilibrium at the home pose with an initial twist and run for 5 s with RK4 at 0.1 ms. With a small twist all cables stay taut and the trajectories agree to 10^{-11} m and 4×10^{-11} rad, or 6×10^{-9} of the motion. With a large twist, 0.8 m/s upward and 1 rad/s about x, cables go slack and tighten again repeatedly, slack in 60 436 of the 400 008 samples of the eight cables, and the agreement is still 8×10^{-12} m and 1.4×10^{-11} rad, against a motion of 59 mm and 0.09 rad. The specification asked for 1%.

4 Cost of a run and planning of the campaign

On one core of this workspace a 16 s run on T1 with the compiled plant at 62.5 μs, the sensors, the logging and the verification controller takes 11.7 s, that is 0.73 s of computing per simulated second; at 125 and 250 μs it takes 7.8 and 6.2 s. The prediction model of the estimators costs 55 μs for one RK4 step, 274 μs for a step with its Jacobians when interpreted, and 46 μs when compiled to C, and the 17 modes of the IMM together take 0.28 ms per cycle compiled. Run at 1 kHz, the IMM predictions alone thus add 0.28 s per simulated second, before the filter updates and the MPC, which do not exist yet and are timed in Phase 3.

From these figures a run of the full method should cost between 25 and 65 s of computing; the range is an estimate, not a measurement, because two of its components are still to be built. The specification asks for at least 100 runs for each of 40 single fault scenarios and the fault free case, which is 4100 runs, or 28 to 74 core hours, for one method under one condition. The proposed method under the two trajectories and the two noise levels, four baselines and three ablations make about ten blocks of that size, so the full campaign is in the order of 300 to 750 core hours. This workspace has two cores and does not keep jobs running unattended between sessions, so it cannot carry a campaign of that size; it can carry the calibration and the tests of Phase 3 and a reduced pilot campaign. For the full campaign a machine with eight or more cores that can run for several days without supervision is needed, for instance a university server. Two levers will reduce the cost once Phase 3 has timed the full method: running the IMM at 200 to 500 Hz, which the specification allows, and compiling the filter updates as the predictions are compiled here.

5 Carried into Phase 3

The verification controller turned the platform over after an upper cable break, which the planner of the study is designed to prevent, and this raises a point that the specification does not settle yet: when a run counts as a lost platform. We will fix a termination rule before any result is produced, for instance a tilt beyond 30°

or a drum leaving its window, so that the failed runs of the baselines end in a defined state rather than in an arbitrary flight. Phase 3 also replaces the verification controller by the estimators and the controller of the methods, tunes them on calibration runs only, and times them, which closes the cost estimate of Section 4.

References

Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11 (1):1–36, 2019. doi: 10.1007/s12532-018-0139-4.

Bosch Rexroth AG. *Synchronous Servomotors MSK: Project Planning Manual.* Bosch Rexroth AG, 12 edition, 2018. DOK-MOTOR*-MSK********-PR12-EN-P, R911296289; MSK030C data in Table 4-5. The values used are identical in edition 05 (2006), titled Rexroth IndraDyn S MSK Synchronous Motors.

Ernst Hairer and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems.* Springer, Berlin, 2 edition, 1996. doi: 10.1007/978-3-642-05221-7.

K. H. Hunt and F. R. E. Crossley. Coefficient of restitution interpreted as damping in vibroimpact. *Journal of Applied Mechanics*, 42(2):440–445, 1975. doi: 10.1115/1.3423596.

Werner Kraus, Philipp Miermeister, Valentin Schmidt, and Andreas Pott. Hybrid position-force control of a cable-driven parallel robot with experimental evaluation. *Mechanical Sciences*, 6:119–125, 2015. doi: 10.5194/ms-6-119-2015.

Andreas Pott, Hendrik Mütherich, Werner Kraus, Valentin Schmidt, Philipp Miermeister, and Alexander Verl. IPAnema: A family of cable-driven parallel robots for industrial applications. In Andreas Pott and Tobias Bruckmann, editors, *Cable-Driven Parallel Robots*, volume 12 of *Mechanisms and Machine Science*, pages 119–134. Springer, 2013.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.

Pauli Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.